

What is Odoo ?

Odoo is an open-source ERP platform that integrates various business applications like CRM, Sales, Inventory, HR, and more into a single system.



Odoo Editions

- **Community Edition** : Free, open-source version.
- **Enterprise Edition** : Paid version with extra features and support.

Odoo Structure

- **Backend** : Python, PostgreSQL, XML
- **Frontend** : QWeb (HTML/XML), JavaScript
- **ORM** : Object-Relational Mapping to manage the database using Python classes.

Installing Odoo on Ubuntu

Step 1 : System Update : `sudo apt update && sudo apt upgrade -y`

Step 2 : Install Required Packages

```
sudo apt install git python3-pip build-essential wget python3-dev python3-venv python3-wheel libxslt-dev
libzip-dev libldap2-dev libsasl2-dev python3-setuptools node-less libjpeg-dev libpq-dev gcc g++ -y
```

Step 3 : PostgreSQL Installation

- `sudo apt install postgresql -y`
- `sudo -u postgres createuser --createdb --username postgres --no-createrole --no-superuser odoo15`

Step 4 : Install Wkhtmltopdf

- `sudo apt install xfonts-75dpi xfonts-base -y`
- `wget https://github.com/wkhtmltopdf/packaging/releases/download/0.12.6-1/wkhtmltox_0.12.6-1.bionic_amd64.deb`
- `sudo apt install ./wkhtmltox_0.12.6-1.bionic_amd64.deb`

Step 5 : Download Odoo Source Code

- `git clone https://github.com/odoo/odoo --depth 1 --branch 15.0 --single-branch odoo15`
- `cd odoo15`

Step 6 : Create Python Virtual Environment

- `python3 -m venv odoo-venv`
- `source odoo-venv/bin/activate`
- `pip3 install wheel`
- `pip3 install -r requirements.txt`

Step 7 : Run Odoo Server

`./odoo-bin -d odoo15 -i base --addons-path=addons --db-filter=odoo15`

Lab 3 : Odoo Basics

Understanding Odoo Components

- **Models** : Define database structure using Python classes.
- **Views** : Define the user interface (form, list, kanban).
- **Menus** : Add navigational menus.
- **Security** : Manage access rights using ACL and record rules.

Practical Example : Student Model

Model Definition

```
from odoo import models, fields

class Student(models.Model) :

    _name = 'student.student'

    _description = 'Student'

    name = fields.Char(string='Student Name', required=True)

    age = fields.Integer(string='Age')

    student_class = fields.Char(string='Class')
```

Views Definition

```
<record id="view_student_form" model="ir.ui.view">

    <field name="name">student.form</field>

    <field name="model">student.student</field>

    <field name="arch" type="xml">

        <form string="Student">

            <sheet>

                <group>

                    <field name="name"/>

                    <field name="age"/>

                    <field name="student_class"/>

                </group>

            </sheet>

        </form>

    </field>

</record>
```

```

        </sheet>

    </form>

</field>

</record>

<record id="view_student_tree" model="ir.ui.view">

    <field name="name">student.tree</field>

    <field name="model">student.student</field>

    <field name="arch" type="xml">

        <tree string="Students">

            <field name="name"/>

            <field name="age"/>

            <field name="student_class"/>

        </tree>

    </field>

</record>

```

Menu Items

```

<menuitem id="menu_student_root" name="Student Management"/>

<menuitem id="menu_student" name="Students" parent="menu_student_root" action="action_student"/>

```

Security

Create **ir.model.access.csv**

id,name,model_id :id,group_id :id,perm_read,perm_write,perm_create,perm_unlink

access_student,access.student,model_student_student,base.group_user,1,1,1,1

Explanation of ir.model.access.csv

1. File Purpose :

Defines **which user groups** can access **which models** and what actions (CRUD) they are allowed to perform.

2. CSV Columns :

- **id** : Unique technical ID for the access rule.
- **name** : Display name of the access rule.
- **model_id:id** : The model this rule applies to (example : `student.student` model).
- **group_id:id** : The user group this rule is for (example : `base.group_user` means Employees).
- **perm_read** : 1 = can read records, 0 = cannot.
- **perm_write** : 1 = can edit records, 0 = cannot.
- **perm_create** : 1 = can create records, 0 = cannot.
- **perm_unlink** : 1 = can delete records, 0 = cannot.

3. Our Example Line :

`access_student,access.student,model_student_student,base.group_user,1,1,1,1`

4. Meaning :

- **ID** : `access_student` (unique internal ID)
- **Name** : `access.student` (display name)
- **Model** : `student.student` (applies to Student model)
- **Group** : `base.group_user` (applies to Employees)
- **Permissions** :

- Read : Yes
- Write : Yes
- Create : Yes
- Delete : Yes

5. Effect :

Employees can **fully manage** (read, create, update, delete) all student records.

6. Example of Read-Only Rule :

`access_student,access.student,model_student_student,base.group_user,1,0,0,0`

👉 This means employees can only **read** student records, no create, edit, or delete allowed.

Lab 4 : Advanced Development

Relationships

- **Many2one** : One student belongs to one class.
- **One2many** : One class has many students.
- **Many2many** : Many students can enroll in many subjects.

Example : Many2one

```
class Classroom(models.Model) :
```

```
    _name = 'student.class'
```

```
    name = fields.Char(string='Class Name')
```

```
class Student(models.Model) :
```

```
    _name = 'student.student'
```

```
    name = fields.Char(string='Student Name')
```

```
    class_id = fields.Many2one('student.class', string='Class')
```

Computed Fields

A **computed field** is a field whose value is automatically **calculated by Odoo**, based on other fields or logic you define.

Key Features:

- The value is **not stored manually**.
- It can be **stored or dynamic**:
 - **store=True** → Odoo saves it in the database (can be searched/sorted).
 - **store=False** (default) → Computed at display time only.
- Requires a **compute method** decorated with **@api.depends**.

```
from odoo import api

@api.depends('subject_ids.grade')

def _compute_average_grade(self) :

    for record in self :

        grades = [subject.grade for subject in record.subject_ids]

        record.average_grade = sum(grades) / len(grades) if grades else 0
```

Scheduled Actions

- A **Scheduled Action** (cron job) in Odoo is a task that **automatically executes** at specified time intervals (e.g., every day, every hour).
- Useful for **background processes** like :
 - Sending automatic reminders.
 - Cleaning up old data.
 - Generating reports.

Key Features :

- Runs automatically based on a **cron schedule**.
- Can execute Python methods from your models.
- Configured from **Settings > Technical > Automation > Scheduled Actions**.
- Defined in **Python code** and optionally registered via XML.

1. Python Method Example

```
from odoo import models, fields
```

```
class Student(models.Model):
```

```
    _name = 'student.student'
```

```
    name = fields.Char(string='Name')
```

```
def send_daily_student_report(self):

    # Example process: Print all student names

    students = self.search([])

    for student in students:

        _logger.info(f'Student: {student.name}')
```

2. XML Configuration

```
<odoo>

<record id="ir_cron_student_report" model="ir.cron">

    <field name="name">Daily Student Report</field>

    <field name="model_id" ref="model_student_student"/>

    <field name="state">code</field>

    <field name="code">model.send_daily_student_report()</field>

    <field name="interval_number">1</field>

    <field name="interval_type">days</field>

    <field name="numbercall">-1</field> <!-- -1 means unlimited calls -->

    <field name="active">True</field>

</record>

</odoo>
```

- **name** : The title of the scheduled action (for identification).
- **model_id** : The model where the Python method is located.
- **state** : Must always be **code** for Python execution.
- **code** : Python instruction to run (example: `model.send_daily_student_report()`).
- **interval_number** : The frequency number (like 1).
- **interval_type** : Time unit (minutes, hours, days, weeks, months).
- **numbercall** : Number of allowed executions:
 - **-1** → unlimited
 - **1** → execute only once
- **active** : Whether the scheduled action is enabled (**True**) or not.

Types of Intervals

- `interval_type = 'minutes'` → every X minutes.
- `interval_type = 'hours'` → every X hours.
- `interval_type = 'days'` → every X days.

How to View and Edit in Odoo

1. Go to **Settings > Technical > Automation > Scheduled Actions**.
2. Search for your scheduled job.
3. You can change the frequency, enable/disable it, or run it manually.

Notes:

- Scheduled actions are managed by the **Odoo worker process**.
- If using a multi-worker production setup, make sure workers are configured to handle cron jobs.
- The `ir.cron` records can be created via XML or manually from the Odoo UI.

Lab 5 : Odoo API Integration

Connect to Odoo via XML-RPC

```
import xmlrpc.client

url = 'http://localhost:8069'

db = 'odoo15'

username = 'admin'

password = 'admin'

common = xmlrpc.client.ServerProxy(f'{url}/xmlrpc/2/common')

uid = common.authenticate(db, username, password, {})

models = xmlrpc.client.ServerProxy(f'{url}/xmlrpc/2/object')
```

```
# Create a student
```

```
models.execute_kw(db, uid, password, 'student.student', 'create', [{ 'name' : 'Ali', 'age' : 18 }])
```

Lab 6 : Building REST API in Odoo

Odoo Controller

```
from odoo import http
```

```
from odoo.http import request
```

```
class StudentAPI(http.Controller) :
```

```
    @http.route('/api/students', auth='public', type='json', methods=['GET'])
```

```
    def get_students(self) :
```

```
        students = request.env['student.student'].sudo().search([])
```

```
        return [ {'id' : s.id, 'name' : s.name, 'age' : s.age} for s in students]
```

Lab 7 : Mobile App Integration

- Use **React Native / Flutter** to fetch data from `/api/students`.
- Use **Axios** to perform GET requests to Odoo.

Example React Native Code

```
axios.get('http://localhost:8069/api/students')
```

```
.then(response => console.log(response.data))
```

```
.catch(error => console.error(error));
```

Lab 8 : Deployment & Security

- Deploy Odoo with Nginx reverse proxy.
- Configure SSL using Let's Encrypt.
- Enable database backups.
- Set strong password policies.

Lab 9 : Mini Project

Project : Bookstore ERP

Modules :

- Book Management
- Customer Management
- Sales Orders
- Stock Reports
- PDF Invoice Generation

Deliverables :

- Full custom Odoo module.
- REST API to connect mobile apps.
- Deployment on a production server.